Liste, tuple, stringhe e altri tipi di dati

Table of contents

Liste, tuple, stringhe e altri tipi di dati 2
- Liste . . . 2
- Creazione di una lista . . . 2
- La funzione `range` . . . 3
- Indicizzare una lista . . . 4
- Aggiungere ed eliminare elementi . . . 5
- Le liste come collezioni MUTABILI: cosa succede in memoria? . . . 7
- Il concetto di Alias . . . 7
- Uguaglianza fra liste . . . 9
- Recuperare più elementi da una lista: lo slicing . . . 10
- Creare copie di una lista . . . 11
- Shallow copy . . . 11
- Deep Copy . . . 15
- Principali metodi e funzioni per le liste . . . 16

TUPLE 17
- TUPLE . . . 17
- Slicing nelle tuple . . . 18

DIZIONARI 18
- Dizionari - creazione . . . 19
- Dizionari - associare un valore ad una chiave . . . 19
- Dizionari - recuperare il valore associato ad una chiave . . . 19
- View objects . . . 20
- Dizionari - metodi principali . . . 22
- Dizionari - esercizi . . . 22

SET 23
- Set: collezione mutabile di elementi non mutabili . . . 24
- Set: altre caratteristiche . . . 24
- Set: funzioni e metodi principali . . . 24

Stringhe **25**
Stringhe: metodi e funzioni utili 26
Esempi 27

Esercizi **27**

Liste, tuple, stringhe e altri tipi di dati

Liste

Le liste sono **sequenze ordinate e modificabili** che raggruppano vari oggetti insieme. Per creare una lista, si usa un'assegnazione con una sequenza di oggetti separati da virgole tra parentesi quadre (`[]`):

```
# Define an empty list
empty = []


# Define a list of numbers
numbers = [1, 2, 3, 100]



# Define a list of strings
superheroes = ["batman", "superman", "spiderman"]


# Define a list of objects with different data types
mixed_types = ["Hello World", [4, 5, 6], False]
```

Creazione di una lista

Alcuni modi per creare una lista

- Lista vuota:

```
a = []
# oppure
b = list()
```

- Lista con elementi:

```
a = [1, 2, 3]
# oppure
b = ['a', 'ciao', 3]
```

- Lista da un'altra stuttura dati:

```
lista = list("ciao")     # ['c', 'i', 'a', 'o']
```

- Mediante la funzione `range`

La funzione `range`

La funzione `range()` restituisce un oggetto iterabile che produce una sequenza di numeri interi. La sintassi è `range(start, stop, step)`

- `start`: valore iniziale della sequenza (incluso). Default: 0
- `stop`: valore finale (escluso)
- `step`: incremento tra un elemento e il successivo. Default: 1

L'oggetto `range` non è una lista, ma immagazzina i parametri essenziali per la creazione di una lista di interi (il primo elemento, lo step, l'ultimo elemento). L'oggetto `range` può essere convertito in una lista tramite il comando `list()`[1].

```
a1 = list( range(10))
# start==0, stop==10 (escluso), step==1
#[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

a2 = list( range (3, 10))
# start==3, stop==10 (escluso), step==1
#[3, 4, 5, 6, 7, 8, 9]

a3 = list( range (3, 10,2))
# start==0, stop==10 (escluso), step==2
#[3, 5, 7, 9]
```

https://docs.python.org/2/library/functions.html#range

[1] tecnicamente `list` è una classe built-in, come vedremo in seguito

Indicizzare una lista

Una **lista** è un contenitore (una *collection*) che consente di gestire un insieme di valori, sia come blocco unico sia accedendo ai singoli elementi uno alla volta. Gli elementi di una lista non devono necessariamente essere omogenei: possiamo combinare numeri interi, numeri decimali (float), stringhe e persino altre liste.

Possiamo accedere al singolo elemento di una lista identificandolo mediante **l'indice di posizione**. L'indice di posizione vale 0 per il primo elemento e vale `len(lista) - 1` per l'ultimo elemento.

Mediante l'indicizzazione possiamo accedere ai singoli elementi sia per leggerne il valore, sia per modificarlo.

Se cerchiamo di accedere ad un indice inesistente otteniamo un errore:

`IndexError: list index out of range`

```
# Indicizzare
a = [1,2,3]
i=0

print(a[i]) # primo elemento
print(a[len(a)-1]) # ultimo elemento


i=1
# modifico il secondo elemento 'in place',
# cioè sulla lista stessa, senza crearne una nuova
a[i] = 10
print(a[i]) # stampo il secondo elemento

#Se cerchiamo di accedere ad un indice inesistente otteniamo un errore:
print(a[5])
#`IndexError: list index out of range`

# NB anche print(a[3]) produce lo stesso errore: perché?
```

Possiamo anche utilizzare una sintassi semplificata che permette di riferirsi agli elementi della lista a partire dall'ultimo. L'ultimo elemento avrà indice `-1`, il penultimo `-2` e così via.

Aggiungere ed eliminare elementi

Le 'operazioni' richiamate su un oggetto mediante l'operatore punto (`.`) si chiamano METODI - approfondiremo meglio questo concetto più avanti.

Il metodo `append()` consente di aggiungere un singolo elemento alla fine della lista.

Il metodo `extend()` permette di aggiungere alla lista tutti gli elementi di un'altra lista, uno dopo l'altro.

```
a = [1, 2, 3]

b = [4, 5, 6]


a.append(10)

a.extend(b)

print(a) #  [1, 2, 3, 10, 4, 5, 6]
```

L'operazione `a.extend(b)` **aggiunge** tutti gli elementi di `b` ad `a`. Possiamo ottenere lo stesso risultato aggiungendo gli elementi uno per uno, ad esempio con un ciclo:

```
while i<len(b):
  el = b[i]
  a.append(el)
```

Questa modalità ci consente di avere maggiore controllo sull'aggiunta degli elementi. Ad esempio, possiamo scegliere di aggiungere solo i valori pari.

L'operazione

```
 c = a + b
```

concatena gli elementi delle liste `a` e `b`, creando una **nuova lista** `c`. Le liste originali `a` e `b` non vengono modificate.

Per eliminare un elemento da una lista, dato l'indice, posso usare `del`, `remove` o `pop`

```
a = [1, 2, 3, 4, 5, 3, 10]
a.remove(3)  # Rimuove la PRIMA occorrenza di 3
# se il valore cercato non esiste, segnala un errore


a = [1, 2, 3, 4, 5, 3, 10]
a.pop(3)  # Rimuove l'elemento all'indice 3 (il 4 in questo caso)
a.pop()  # Rimuove l'ultimo elemento della lista (il 10 in questo caso)


a = [1, 2, 3, 4, 5, 3, 10]

del a[3] # Rimuove l'elemento all'indice 3 (il 4 in questo caso)
```

Sia `del` sia `pop()` permettono di eliminare elementi da una lista, ma con una differenza sostanziale.

- `del a[i]` rimuove l'elemento di indice `i` e non restituisce un valore
- `a.pop(i)` rimuove l'elemento di indice `i` e lo restituisce:

```
returned_value = a.pop(i)
print(returned_value)
```

- `remove` invece non restituisce nessun valore. Si noti che `remove` accetta in ingresso un valore da cercare nella lista, mentre `del()` e `pop()` accettano un indice.

Come detto, gli elementi di una lista non devono necessariamente essere omogenei: possiamo combinare numeri interi, numeri decimali (float), stringhe e persino altre liste.

`a= [1,  [10, 20, 30], 3]`

In questo caso, 'a[1] è il secondo elemento della lista

`a[1] == [10, 20, 30]`

Posso indicizzare l'elemento interno come indicizzo una normale lista

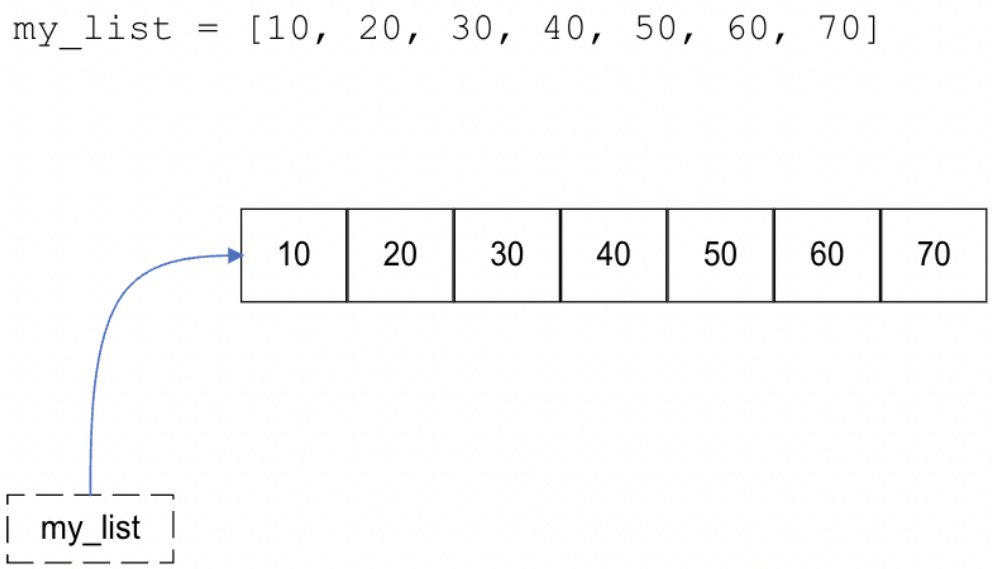
```
a= [1,  [10, 20, 30], 3]

print(a[1][0] ) # 20

b= a[1][2]
print(a[1][2] ) # 30
```

In questo caso `a[1]` indca la lista `[10, 20, 30]` e `a[1][2]` indica l'elemento di indice 2 della lista `[10, 20, 30]`

Le liste come collezioni MUTABILI: cosa succede in memoria?

`my_list = [10, 20, 30, 40, 50, 60, 70]`

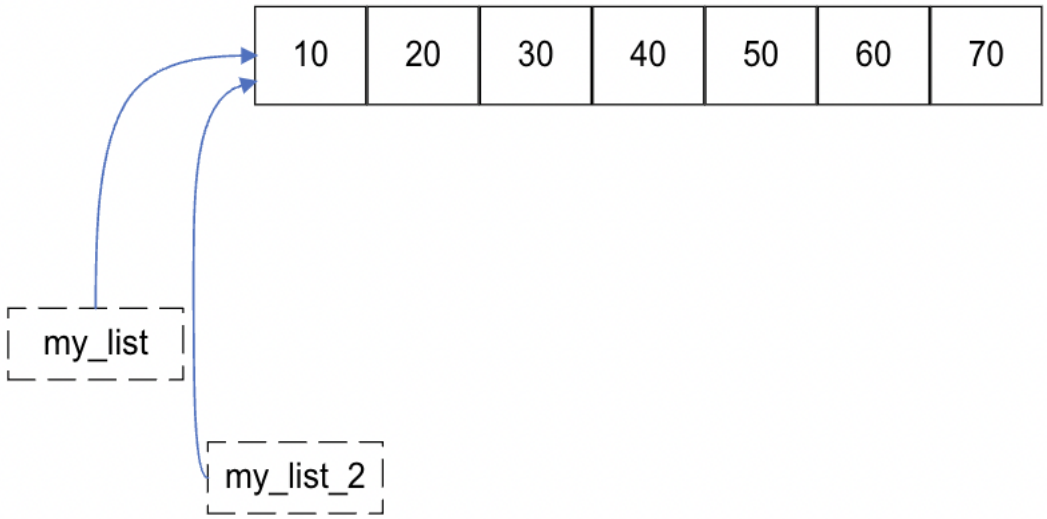
Quando creiamo una lista, gli elementi vengono memorizzati in un'area di memoria e alla variabile viene assegnato un riferimento a questa posizione di memoria, proprio come avviene per le altre variabili di tipo semplice.



Il concetto di Alias

L'operazione `my_list_2 = my_list` assegna a `my_list_2` lo stesso riferimento in memoria di `my_list`. Non viene creata una nuova lista, ma semplicemente un altro nome che punta alla stessa lista in memoria. Viene stabilito un collegamento (*binding*) tra la variabile `my_list_2` e l'oggetto originale.

Questo spiega perché l'istruzione `my_list[0] = 100` *sembra* modificare entrambe le liste. In realtà, c'è una sola lista. Poiché entrambe le variabili fanno riferimento alla stessa area di memoria, ogni modifica a un elemento della lista è visibile da entrambe le variabili.



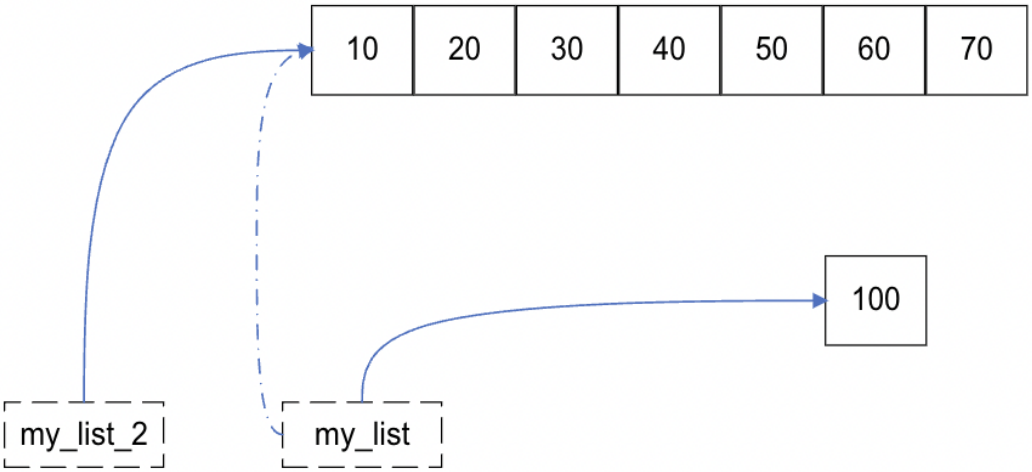
E' semplice verificare che i due identificatori `my_list` e `my_list_2` si riferiscono allo stesso oggetto in memoria:

```
my_list = [10, 20, 30, 40, 50, 60, 70]
my_list_2 = my_list
print(id(my_list))
print(id(my_list_2))
```

Consideriamo le seguenti istruzioni

```
my_list = [10, 20, 30, 40, 50, 60, 70]
my_list_2 = my_list
my_list = 100
print(my_list)
print(my_list_2)
print(id(my_list))
print(id(my_list_2))
```

L'ultima istruzione fa sì che `my_list` non punti più alla lista originale, ma ora faccia riferimento al valore `100`.



Uguaglianza fra liste

Due liste sono considerate uguali se contengono gli *stessi elementi* nello *stesso ordine*

```
l1 = [1,2,3]
l2 = [1,2,3] # stessa sequenza di valori di l1
l3 = [3, 1, 2] # stessi valori, ordine diverso

print(l1==l2) #True
print(l1==l3) #False
```

tuttavia, non per questo sono lo **stesso oggetto** in memoria.

Per verificare invece se due variabili puntano allo stesso oggetto (stessa locazione in memoria), si usa l'operatore `is` (o si confrontano gli `id`):

```
print(l1 is l2)            # False: non è lo stesso oggetto
print(hex(id(l1)), hex(id(l2))     # due valori diversi, perché sono due oggetti distinti
```

Recuperare più elementi da una lista: lo slicing

Un'esigenza comune quando si lavora con le liste è quella di estrarre una porzione, o sottolista, da una lista esistente. Questo è possibile grazie a un'operazione chiamata slicing, che utilizza la seguente sintassi:

`lista[start:stop:step]`

La parte `[start:stop:step]` è nota come operatore di slicing. La sua sintassi prevede una coppia di parentesi quadre e fino a tre indici opzionali: `start`, `stop` e `step`,

- `start`: indica l'indice da cui iniziare lo slicing. L'elemento a questo indice **è incluso** nel risultato.
- `stop`: indica l'indice in cui terminare lo slicing. L'elemento a questo indice **non è incluso** nel risultato.
- `step`: rappresenta il numero di elementi da saltare ad ogni passo

L'istruzione `lista[2:10]` estrae una sottolista che include gli elementi a partire dall'indice 2 fino all'indice 10, **escludendo** l'elemento all'indice 10

L'istruzione `lista[2:10:3]` estrae una sottolista a partire dall'indice 2 fino all'indice 10 (escluso), procedendo con uno step di 3. In questo caso verranno selezionati gli elementi agli indici 2, 5 e 8, ovvero `lista[2]`, `lista[5]` e `lista[8]`.

Tutti e tre gli indici sono *facoltativi* e, se omessi, assumono i seguenti valori predefiniti:

- `start`: 0 (inizio della sequenza)
- `stop`: lunghezza della sequenza
- `step`: 1 (nessun salto, elementi consecutivi)

Esempio (si riportano i casi più comuni)

```
#
lista2 = lista[2:10:3] # dall'indice 2 all'indice 10 (escluso), con step di 3
lista2 = lista[2:10] # dall'indice 2 all'indice 10 (escluso), con step di 1
lista2 = lista[2:] # dall'indice 2 al termine della lista, con step di 1
lista2 = lista[:10] # dall'inizio della lista all'indice 10 (escluso)

lista2 = lista[:]
# tutti gli elementi della lista
# (omettendo `start`, `stop` e `step`, questi assumono i valori di defaul)

lista2 = lista[10:2:-1] # dall'indice 10 all'indice 2 (escluso), con step di -1
lista2 = lista[::-1]  # Tutti gli elementi in ordine inverso
```

Tabella riassuntiva per lo slicing:

Sintassi	Descrizione	Esempio / Note
`lista[start:stop]`	Estrae elementi da `start` a `stop` (escluso)	`lista[2:5]` → elementi in pos. 2,3,4
`lista[start:]`	Estrae dal punto `start` fino alla fine	`lista[2:]` → dal 2° elemento in poi
`lista[:stop]`	Estrae dall'inizio fino a `stop` (escluso)	`lista[:4]` → primi 4 elementi
`lista[:]`	Copia completa della lista	Crea una **copia superficiale**
`lista[start:stop:step]`	Estrae da `start` a `stop` (escluso), saltando di `step` elementi	`lista[1:7:2]` → pos. 1, 3, 5
`lista[::step]`	Estrae tutta la lista con passo `step`	`lista[::2]` → un elemento ogni due
`lista[::-1]`	Estrae la lista al contrario	Inversione rapida
`lista[start::-1]`	Estrae da `start` all'inizio, all'indietro	`lista[4::-1]` → da pos. 4 a 0
`lista[:stop:-1]`	Estrae dalla fine fino a `stop` (escluso), all'indietro	`lista[:2:-1]` → da fine a pos. 3
`lista[start:stop:-1]`	Estrae da `start` a `stop` (escluso), all'indietro	`lista[5:1:-1]` → pos. 5, 4, 3, 2

Creare copie di una lista

Come abbiamo già visto, in Python ad ogni oggetto è associato un **identificatore univoco**, ottenibile tramite la funzione `id()`. Nella maggior parte delle implementazioni di Python (come CPython), la funzione `id()` rende l'indirizzo di memoria in cui l'oggetto è memorizzato.

In Python esistono due modalità principali per creare copie di una lista:

- copia superficiale (**shallow copy**)
- copia profonda (**deep copy**)

Shallow copy

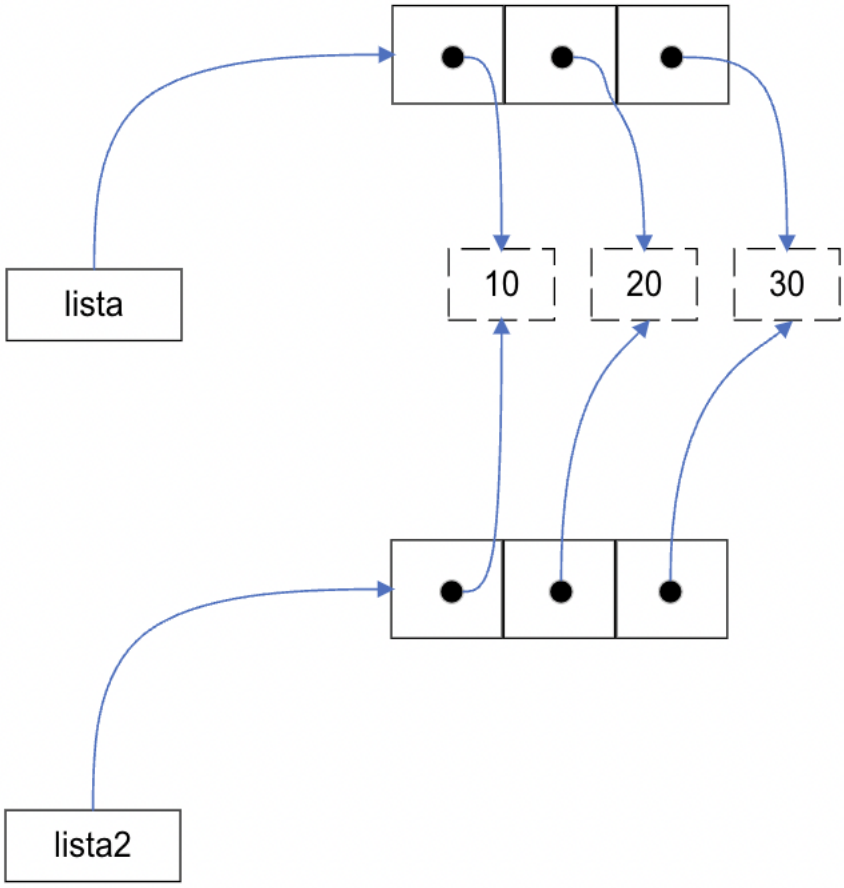
Una copia superficiale di una lista è una **nuova lista che contiene riferimenti agli stessi oggetti presenti nella lista originale**. Python crea un nuovo oggetto lista, ma inserisce al suo interno i riferimenti agli stessi elementi contenuti nella lista di partenza. Questo significa che, **anche se la nuova lista è indipendente come contenitore, gli oggetti al suo interno non sono duplicati**: sono condivisi tra la lista originale e la copia.

L'operazione

```
lista2 = lista[:]
```

crea una copia superficiale della lista. Esaminiamo alcuni esempi in dettaglio.

```
lista = ['paperino', 'pippo', 'topolino']
lista2 = lista[:]
print(lista == lista2) # True: il contenuto è uguale
print(id(lista[0]) == id(lista2[0])) # True: il primo elemento è lo stesso
print(id(lista) == id(lista2)) #False: le due liste sono oggetti diversi
```



Esistono almeno tre modi per creare una copia superficiale di una lista in Python:

```
# L'operatore di slicing [:]
lista2 = lista[:]
```

```
# Il metodo .copy() delle liste
lista2 = lista.copy()

# La funzione `copy()` del modulo copy

import copy
lista2 = copy.copy(lista)
```

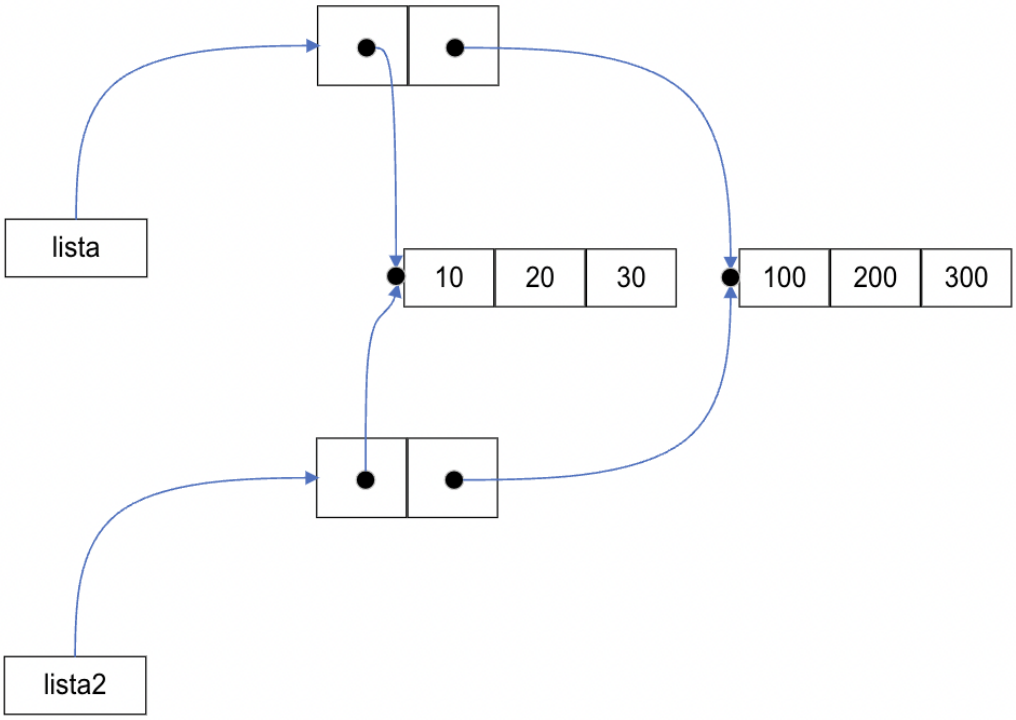
I tre modi sono sostanzialmente uguali per quanto rigrarda le liste. La funzione ed il metodo `copy` saranno utili quando faremo delle copie superficiali di oggetti diversi dalle liste, per i quali lo slicing non è applicabile.

Gli effetti della copia superficiale emergono nettamente quando gli elementi della lista sono a loro volta elementi *mutabili*, per esempio altre liste

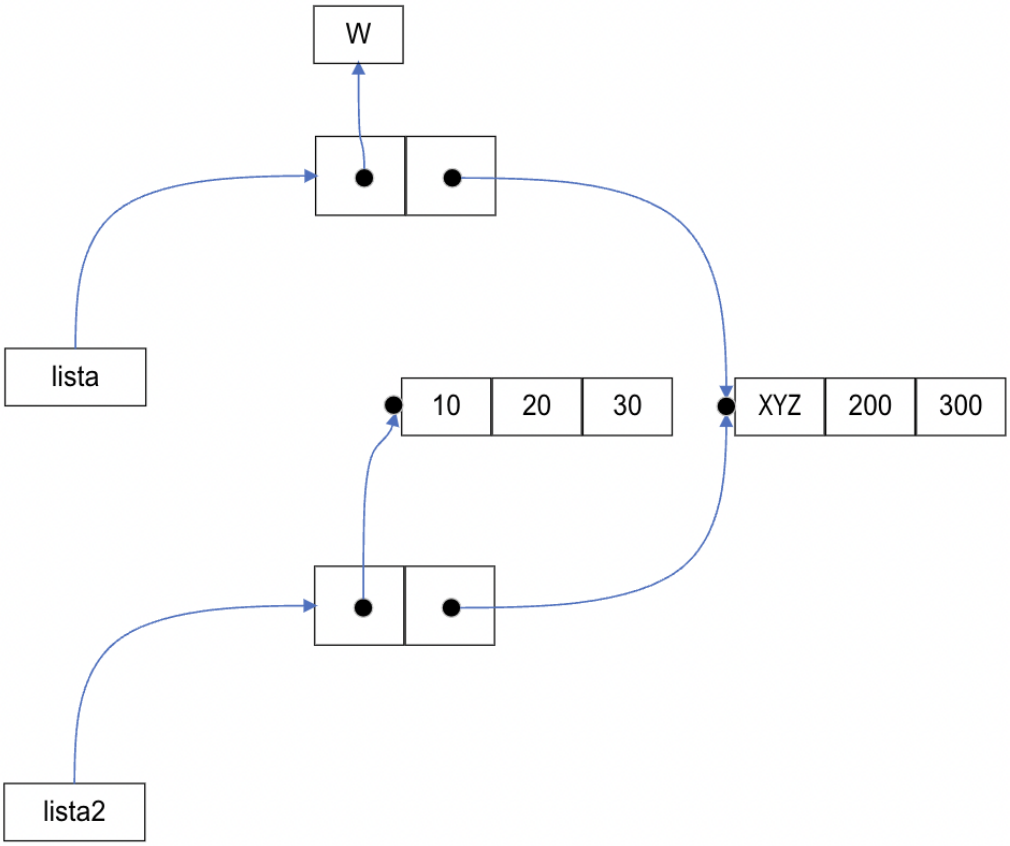
```
lista = [ [10, 20, 30], [100, 200, 300]]
lista2 = lista[:]
lista[0] = `W`
lista[1][0] = 'XYZ'
```

L'esempio mostra che le due liste **non** sono indipendenti: il primo elemento cambia solo per `lista`; il secondo elemento cambia per entrambe.

```
lista = [ [10, 20, 30], [100, 200, 300]]
lista2 = lista[:]
```



```
lista[0]='W'
lista[1][0] = 'XYZ'
```

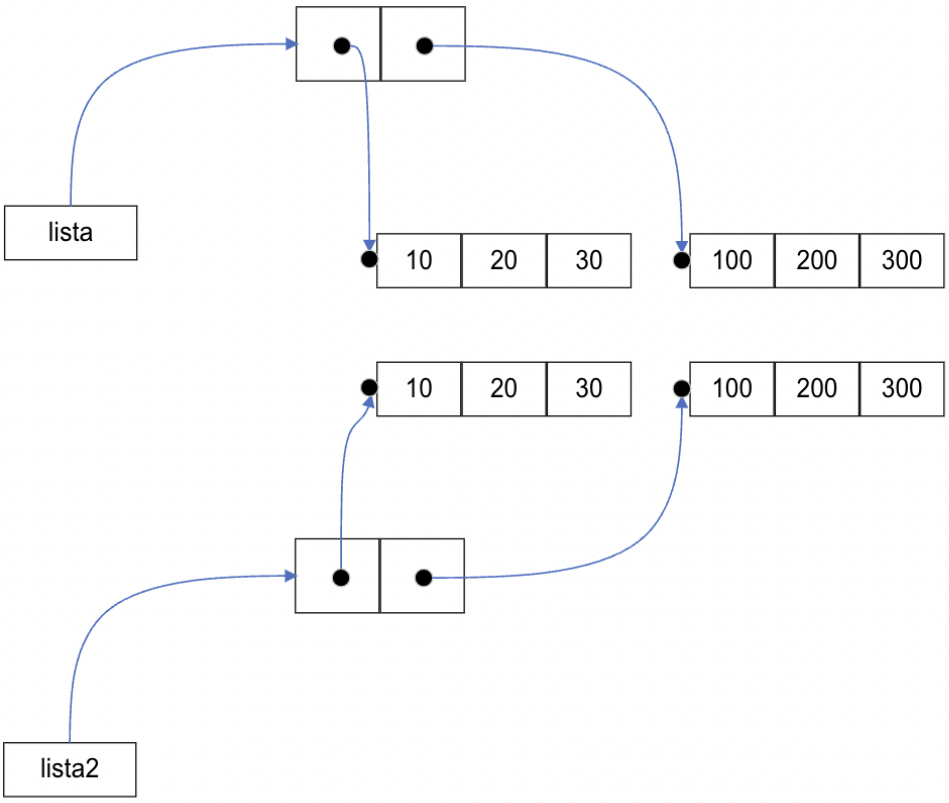


Deep Copy

Una **deep copy** è una copia completa e indipendente di una lista (in genere, di un oggetto), compresi tutti gli oggetti annidati al suo interno. Le modifiche fatte alla copia non influenzano l'originale (e viceversa).

```
import copy
lista = [ [10, 20, 30], [100, 200, 300]]
lista2 = copy.deepcopy(lista)

lista[0] = `W`
lista[1][0] = 'XYZ'
```

```
lista = [ [10, 20, 30], [100, 200, 300]]
lista2 = copy.deepcopy(lista)
```



https://docs.python.org/3/tutorial/datastructures.html#more-on-lists

Principali metodi e funzioni per le liste

Metodo / Funzione	Tipo	Descrizione	Esempio
`append(x)`	metodo	Aggiunge l'elemento `x` alla fine della lista	`L.append(4)`
`extend(iter)`	metodo	Estende la lista aggiungendo tutti gli elementi di `iter`	`L.extend([4,5])`
`remove(x)`	metodo	Rimuove la prima occorrenza di `x`; solleva `ValueError` se non trovato	`L.remove(2)`
`clear()`	metodo	Rimuove tutti gli elementi	`L.clear()`

Metodo / Funzione	Tipo	Descrizione	Esempio
`copy()`	metodo	Restituisce una copia superficiale (shallow copy)	`M = L.copy()`
`len(L)`	funzione	Restituisce il numero di elementi	`len(L)`
`sum(L)`	funzione	Somma tutti gli elementi numerici	`sum([1,2,3])`
`min(L)`	funzione	Restituisce il valore minimo	`min([1,2,3])`
`max(L)`	funzione	Restituisce il valore massimo	`max([1,2,3])`
`any(L)`	funzione	`True` se almeno un elemento è `True` (come in OR)	`any([0, '', 5]) # True`
`all(L)`	funzione	`True` se tutti gli elementi sono `True` (come in AND)	`all([1, 'a', True]) # True`

TUPLE

TUPLE

Le **tuple** sono simili alle liste, ma sono sequenze **immutabili**. Ciò significa che non è possibile modificarle dopo la creazione.

Non è possibile utilizzare il metodo `append` nè altri metodi che tentino di modificare la tupla. Le regole di indicizzazione sono le medesime viste per le liste, ma posso solo *leggere* i valori nella tupla, non modificarli.

I due metodi principali per creare una tupla in Python sono i seguenti:

- Specificare gli elementi separati da virgole all'interno di parentesi tonde, ad esempio: `t=(a, b, c)`
- Utilizzare la funzione built-in `tuple()` per convertire un'altra sequenza (come una lista) in una tupla: `t= tuple([a, b, c])`

```
t = (1,2,3)

# t.append(4) # errore
# t[1] = 10 # errore

print(t[0]) # OK
```

Si utilizzano le tuple quando occorre essere sicuri che i dati non siano modificati e quando occorre più velocità nell'accesso ai dati.

Slicing nelle tuple

Proprio come con le liste, anche con le tuple è possibile effettuare indicizzazione e suddivisione in sezioni (slicing):

```
t =(0, 11, 22, 33, 44, 55)

print(t[0]) # 0

print(t[0:4]) # (0, 11, 22, 33)
```

Esempio: aggiungere un elemento ad una tupla

```
t = (1,2,3)

# t.append(4) # errore
# non posso modificare una tupla esistente:
# devo crearne una nuova

i = 0
tmp = []
while i<len(t):
  tmp.append(t[i])
  i += 1
tmp.append(4)

t = tuple(tmp)

print(t)
```

DIZIONARI

Un dizionario in Python è una struttura dati built-in che memorizza **coppie chiave-valore**. Le chiavi sono univoche e immutabili. Le chiavi dunque possono essere numeri, strighe, tuple ma **non liste**. I valori possono essere di qualsiasi tipo.

Dizionari - creazione

1. parentesi graffe `{}` con coppie chiave: valore, separate da virgole.

```
dizionario = {
    "nome": "Topolino",
    "eta": 30,
    "lingua": "Python"
}
```

2. funzione `dict()`

```
dizionario = dict(nome = Topolino", eta = 30, lingua = Python )
```

Dizionari - associare un valore ad una chiave

L'operazione

```
d[k] =  v
```

associa il valore `v` alla chiave `k` nel dizionario `d`.

Se la chiave `k` non esiste, viene creata una nuova coppia chiave-valore.

Se la chiave esiste già, il valore precedente viene sovrascritto.

è possibile dunque creare un dizionario vuoto e riempirlo successivamente

```
dizionario = dict()
dizionario["nome"] = "Topolino",
dizionario["eta"] = 30,
dizionario["lingua"] = "Python"
```

Dizionari - recuperare il valore associato ad una chiave

L'operazione

```
v = d[k]
```

rende il valore associato alla chiave `k`, se la chiave è presente nel dizionario `d`. Se invece `k` non è presente, Python genera un errore (`KeyError`).

Questo accesso è deterministico e costante in tempo (`O(1)`), ma non è sicuro: se la chiave `k` non è presente, Python solleva un'eccezione di tipo `KeyError`.

Per evitare questo errore, si possono adottare due strategie:

- Verificare esplicitamente la presenza della chiave, con `in`:

```
if k in d:
    v = d[k]
```

- Utilizzare il metodo `get`, che restituisce il valore associato a una chiave, se presente. Se la chiave non esiste, restituisce `None` o un valore di default passato come secondo argomento.

```
v1 = d.get(k1)

v2 = d.get(k2, valore_predefinito)
```

View objects

È possibile recuperare una **vista** delle chiavi, dei valori e delle coppie chiave-valore in un dizionario utilizzando rispettivamente i metodi `.keys()`, `.values()` e `.items()`.

In Python, un **view object** (oggetto vista) è un oggetto che rappresenta dinamicamente la porzione di dati del dizionario sottostante, senza copiarli.

- Se il dizionario viene modificato (aggiunta, rimozione o modifica di voci), la *vista* si aggiorna automaticamente.
- La *vista* è iterabile - si può utilizzare in un ciclo FOR esattamente come una lista (vedere esempi nel capitolo dedicato al ciclo FOR).
- Non supporta l'indicizzazione diretta (es. `d.keys()[0]` genera un errore)
- Implementa operazioni insiemistiche (set-like) quando ha senso; per es. unione (`|`), intersezione (`&`) e differenza (`-`).
- È efficiente in termini di memoria, perché non duplica i dati del dizionario.
- È possibile trasformare la vista in **lista** mediante `list()`

https://docs.python.org/3/library/stdtypes.html#dictionary-view-objects

---

Esempio

Notare la differenza di comportamento tra vista e lista. La vista `a` riflette le modifiche applicate al dizionario. La lista `b` è un oggetto autonomo.

```
d= {'chiave1':1, 'chiave2':2, 'chiave3':3, 'chiave4':4}
a= d.items()
b= list(d.items())
print('\nvista e lista corrispondente')
print('vista->',a)
print('lista->',b)
d['chiave5']=5
print("\nvista e lista corrispondente dopo l'aggiunta di un item")
print('vista->',a)
print('lista->',b)

# vista-> dict_items([('chiave1', 1), ('chiave2', 2), ('chiave3', 3), ('chiave4', 4), ('ch
# lista-> [('chiave1', 1), ('chiave2', 2), ('chiave3', 3), ('chiave4', 4)]
```

La vista `a` è una **vista di sola lettura**: riflette le modifiche fatte sul dizionario, ma non permette di alterarne i valori direttamente. Non supporta né l'indicizzazione né l'assegnamento.

```
print('vista->',a)

# vista-> dict_items([('chiave1', 1), ('chiave2', 2), ('chiave3', 3),
# ('chiave4', 4), ('chiave5', 5)])

a[0]
# TypeError: 'dict_items' object is not subscriptable

a[0] = 0
# TypeError: 'dict_items' object does not support item assignment
```

è bene rimarcare il fatto che le chiavi devono essere oggetti immutabili (tecnicamente, *hashable*)

https://docs.python.org/3.12/glossary.html#term-hashable

Questo requisito è indispensabile per il corretto funzionamento dei meccansmi di indicizzazione dei dizionari.

Non ci sono limitazioni invece per i valori della coppia chiave-valore.

Caution

Da Python 3.6 (implementazione CPython) i dizionari sono strutture dati ordinate. Questo significa che mantengono l'ordine di inserimento delle chiavi: il ciclo FOR scandisce le chiavi nell'esatta sequenza in cui sono state aggiunte. Nelle versioni precedenti a Python 3.6 l'ordine non era definito, quindi, se si vuole garantire compatibilità con interpreti più vecchi, non si deve in alcun modo fare affidamento sull'ordinamento delle chiavi.
Se si utilizza una versione di Python precedente alla 3.6 e si necessita di un dizionario ordinato, si può considerare l'utilizzo di `collections.OrderedDict()`.
https://docs.python.org/3/library/collections.html#collections.OrderedDict

Dizionari - metodi principali

Metodo	Descrizione
`d.get(chiave)`	Restituisce il valore associato alla chiave, oppure `None` se non esiste
`d.keys()`	Restituisce una **vista** delle **chiavi** del dizionario
`d.values()`	Restituisce una **vista** dei **valori**
`d.items()`	Restituisce una **vista** delle **coppie (chiave, valore)**
`d.pop(chiave)`	Rimuove la chiave e restituisce il valore associato
`d.clear()`	Rimuove tutti gli elementi del dizionario
`chiave in d`	Restituisce `True` se la chiave è presente nel dizionario
`len(d)`	Restituisce il numero di coppie chiave-valore nel dizionario

Dizionari - esercizi

Data una `lista`, creare una `lista1` contenente tutti gli elementi che compaiono esattamente una volta e una `lista2` contenente tutti gli elementi che compaiono esattamente due volte

```
lista = [ 1 , 2, 3, 4, 2, 3,3, 4,4,4, 10, 20,20 ]

lista1= []
lista2= []
d = dict()
for k in lista:
    d[k] = d.get (k, 0 ) +1

for k, v in d.items():
    if v == 1:
        lista1.append(k)
    elif v == 2:
        lista2.append(k)
```

```
print('lista', lista)
print('lista1', lista1) # 1, 10
print('lista2', lista2) # 2, 20
```

l'istruzione `d[k] = d.get (k, 0 ) +1`

può essere sostituita con

```
    if k in d:
        d[k] =  d[k] +1
    else:
         d[k] = 1
```

(provate)

SET

I set sono collezioni *non ordinate* e *mutabili* di oggetti. È possibile creare set in diversi modi. Eccone alcui:

- inserendo direttamente gli oggetti fra parentesi graffe
- passando una lista alla funzione `set()`
- creando un set vuoto e poi aggiungendo elementi

```
s1={1,2,3}
a=[10, 20, 30]
s2= set(a)
s3= set()
# NB: s3={} crea un DIZIONARIO vuoto, non un set vuoto
s3.add(100)
s3.add(200)
```

Il set è un insieme di elementi unici. Posso usarlo quindi per rimuovere i duplicati da una lista.

Esempio:

```
a= [1,2,3,4,3,4,3,2,4]
s=set (a)
a= list(s)
```

Set: collezione mutabile di elementi non mutabili

Dire che un set è mutabile significa che l'oggetto stesso può cambiare, cioè è possibile

- aggiungere elementi (es. `s.add(2)`)
- rimuovere elementi (es. `s.remove(2)`)

Non è possibile modificare un elemento esistente, come invece si può fare con le liste

```
s={1,2,3}
s[0]=1
TypeError: 'set' object does not support item assignment
```

Gli elementi di un set devono essere immutabili (hashable). L'insieme è mutabile, ma ogni singolo valore al suo interno deve avere un hash stabile (es. numeri, stringhe, tuple immutabili). Per questo motivo **le liste (i dizionari, ecc.) non possono essere elementi di un set**

Set: altre caratteristiche

I set sono insiemi non ordinati. Non esiste una 'posizione' 0, 1, 2… come nelle liste. Non è possibile dunque indicizzarli

```
print(s[0])
#TypeError: 'set' object is not subscriptabl
```

I set supportano le tipiche operazioni sui set, come **union** (`|`), **intersection** (`&`), **difference** (`-`)

Set: funzioni e metodi principali

Metodo / Funzione	Descrizione	Esempio
`add(x)`	Aggiunge l'elemento `x` al set	`S.add(4)`
`update(iterable)`	Aggiunge tutti gli elementi di `iterable`	`S.update([1,2,3])`
`remove(x)`	Rimuove `x`; solleva `KeyError` se non esiste	`S.remove(2)`
`discard(x)`	Rimuove `x` se presente, altrimenti non fa nulla	`S.discard(2)`
`pop()`	Rimuove e restituisce un elemento arbitrario	`elem = S.pop()`
`clear()`	Rimuove tutti gli elementi	`S.clear()`
`copy()`	Restituisce una copia superficiale (shallow copy)	`T = S.copy()`
`union(*others)`	Restituisce l'unione con altri set	`S.union({3,4})`

Metodo / Funzione	Descrizione	Esempio
`intersection(*others)`	Restituisce l'intersezione con altri set	`S.intersection({2,3})`
`difference(*others)`	Restituisce la differenza da altri set	`S.difference({1})`
`len(S)`	Restituisce il numero di elementi	`len(S)`
`set(iterable)`	Converte un iterabile in set	`set([1,2,3])`
`any(S)`	`True` se almeno un elemento del set è `True`	`any({0, '', 5})`
`all(S)`	`True` se tutti gli elementi del set sono `True`	`all({1, True, 'a'})`

Esempi:

```
# definizione di due set di esempio
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

# unione: tutti gli elementi presenti in A o in B
print("Unione A   B:", A.union(B))           # {1, 2, 3, 4, 5, 6}
print("Unione (operatore |):", A | B)        # {1, 2, 3, 4, 5, 6}

# intersezione: solo gli elementi comuni ad A e B
print("Intersezione A   B:", A.intersection(B))   # {3, 4}
print("Intersezione (operatore &):", A & B)       # {3, 4}

# differenza: elementi in A ma non in B, e viceversa
print("Differenza A - B:", A.difference(B))        # {1, 2}
print("Differenza B - A:", B.difference(A))        # {5, 6}
print("Differenza (operatore -):", A - B, B - A)    # {1, 2} {5, 6}
```

Stringhe

Le stringhe sono sequenze di caratteri che è possibile definire utilizzando virgolette singole, doppie o triple:

```
str1= 'ciao mondo'
str2= "ciao mondo"
str3= """ciao
a
tutti"""

print(str1)
print(str2)
print(str3)
```

Le triple virgolette permettono di scrivere una stringa su più righe di testo

---

Analogamente ad una tupla, la stringa è **immutabile**. Non possiamo cambiare, eliminare o aggiungere caratteri. Possiamo però indicizzare la stringa per accedere al singolo carattere, o applicare lo slicing, analogamente a quanto si fa con le liste o le tuple.

```
# ok!
s1= 'ciao'
print(s1[-1]) #o
s2=s1[1:3]
print(s2) #ia


# Errori!
s1[0]= 'x' # immutabile! sola lettura
s1.add('x') # immutabile!
s1.append('x') # immutabile!
```

Posso unire due stringhe utilizzando l'operatore +. Questo processo viene chiamato *concatenazione*. Nota: NON stiamo modificando la stringa originale, stiamo creando una nuova stringa.

```
s1 = 'ciao'
s1 = s1 + ' ' + 'mondo'
print(s1)  # Output: ciao mondo
```

Stringhe: metodi e funzioni utili

Provate e padroneggiate queste funzioni e metodi!

Funzione/Metodo	Descrizione	Esempio
`+`	Concatenazione	`'ciao' + ' mondo'`
`*`	Ripetizione	`'ciao' * 3`
`len(s)`	Lunghezza della stringa	`len('ciao')`
`str(x)`	Conversione in stringa	`str(123)`
`ord(c)`	Codice Unicode di un carattere	`ord('a')`

Funzione/Metodo	Descrizione	Esempio
`chr(n)`	Carattere da codice Unicode	`chr(97)`
`s.lower()`	Tutto minuscolo	`'CIAO'.lower()`
`s.upper()`	Tutto maiuscolo	`'ciao'.upper()`
`s.capitalize()`	Prima lettera maiuscola	`'ciao mondo'.capitalize()`
`s.title()`	Iniziali maiuscole per ogni parola	`'ciao mondo'.title()`
`s.split(sep)`	Divide la stringa in lista	`'ciao mondo'.split()`
`" ".join(lista)`	Unisce elementi della lista in stringa	`" ".join(['ciao', 'mondo'])`
`s.isdigit()`	Verifica solo cifre	`'12345'.isdigit()`
`s.isalpha()`	Verifica solo lettere	`'abcDEF'.isalpha()`
`s.isalnum()`	Verifica lettere o cifre	`'abc123'.isalnum()`
`s.isspace()`	Verifica solo spazi	`'  '.isspace()`
`s.islower()`	Tutto minuscolo	`'ciao'.islower()`
`s.isupper()`	Tutto maiuscolo	`'CIAO'.isupper()`
`s[i]`	Accesso a singolo carattere	`'ciao'[0]` → `'c'`
`s[start:end]`	Slicing sottostringa	`'ciao mondo'[0:4]` → `'ciao'`

Esempi

```
lista_di_stringhe=['il', 'gatto', 'è', 'seduto', 'sul', 'tetto']
frase = ' '.join(lista_di_stringhe)
print(frase)
print(frase.split())
```

Esercizi

(in progress)